

Sistema de Sinalização Embarcado

ESP32, MicroPython, LEDs, Botão e OLED
SSD1306

Candidato:

Sergio Cordero Calvimontes

Plataforma: Wokwi (Simulador)

Linguagem: MicroPython

Arquitetura: Assíncrona (asyncio)

Visão Geral & Hardware

Objetivo do Projeto

Desenvolver e simular um sistema embarcado responsivo com **ESP32** integrando tópicos de:

- Instrumentação
- Eletrônica
- Lógica de Programação

Placa-Alvo

Espressif ESP32-DevKitC V4

Módulo recomendado: **ESP32-WROOM-32E**.

(Evitar WROVER: usa internamente os GPIOs 16/17 para a PSRAM.)

Conexões dos GPIOs

- **LED Vermelho:** GPIO 2 (resistor 220 $[\Omega]$)
- **LED Verde:** GPIO 4 (resistor 220 $[\Omega]$)
- **Botão pulsador:** GPIO 17 (PULL_DOWN)
- **OLED SCL:** GPIO 25 (I²C *clock*)
- **OLED SDA:** GPIO 16 (I²C *data*)

Funcionamento

LED Vermelho (GPIO 2)

Pisca continuamente a cada **500 [ms]**, em uma tarefa assíncrona própria.

Resposta ao Botão (GPIO 17)

1. Botão solto (LOW):

- LED Verde: **APAGADO**
- OLED: "Boa sorte!"

2. Botão pressionado (HIGH):

- LED Verde: **ACESO**
- OLED: "Consegui"

Duas Tarefas, Nunca Bloqueantes

O programa usa exclusivamente `asyncio` do MicroPython, em vez de um *superlaço* (*superloop*) com `time.sleep()`. As duas tarefas acima rodam como **corrotinas** sob escalonamento cooperativo, e nenhuma bloqueia a outra: o piscar do LED vermelho nunca atrasa a resposta ao botão, nem a leitura do botão interrompe o LED vermelho.

Atualização Dinâmica

OLED: Atualização Dinâmica

O mostrador **não** é redesenhado a cada iteração: é atualizado apenas quando o estado **estável** do botão muda (validado por antirrepique de software, ~ 30 [ms]).

- Evita tráfego I²C desnecessário.
- Elimina a cintilação (*flickering*).
- O antirrepique dispensa filtro RC externo; na simulação Wokwi é mantido como boa prática, já que o botão simulado não tem repique real.

Validação

Estratégia de Testes Isolados

Desenvolvimento com testes progressivos em `tests/`, do mais simples ao mais completo:

1. **LED Vermelho:** ciclo de 500 [ms], depois repetido com `asyncio`.
2. **Botão + LED Verde:** nível lógico e resistor interno de *pull-down*.
3. **OLED I²C:** varredura de endereço (0x3C) e testes gráficos completos.
4. **Integração Final:** execução concorrente em `main.py`.

Conclusão

Requisitos Atendidos

- ✓ LED vermelho: ciclo de 500 [ms], contínuo e não bloqueante.
- ✓ LED verde: reflete o estado do botão (GPIO 17, PULL_DOWN).
- ✓ OLED: mensagem correta por estado, com atualização dinâmica.
- ✓ Execução concorrente via `asyncio`, sem `time.sleep()`.

Links para Acesso:

Simulação no Wokwi

<https://wokwi.com/projects/471528241540407297>

Repositório GitHub

<https://github.com/Argus-Kepheus/cess-uff>